

Product Requirements Document (PRD)

TestSprite Next.js Demo Application

1. Product Overview

Product Name: TestSprite Demo App

Version: 1.0.0

Date: May 2026

Author: TestSprite Demo Team

Purpose

A demonstration Next.js application designed to showcase TestSprite's autonomous testing capabilities across authentication, forms, API routes, and protected content.

2. Target Audience

- Developers evaluating TestSprite
 - QA teams learning autonomous testing
 - Demonstration of Next.js testing workflows
-

3. Features & Requirements

3.1 Authentication System

Description: Users must authenticate to access protected pages.

Requirements:

- Login page accessible at `/login`
- Accepts email and password credentials
- Hardcoded test credentials:
 - Email: `test@example.com`
 - Password: `password123`
- On successful login:
 - Set cookie `auth-token=logged-in`
 - Redirect to `/dashboard`
- On failed login:
 - Display error message: "Use [test@example.com](#) / password123"
 - Remain on login page
- Logout functionality on dashboard:
 - Clear auth cookie
 - Redirect to login page

Test Scenarios:

- Valid credentials → redirect to dashboard
- Invalid credentials → show error

- Logout → clear session, redirect to login
-

3.2 Protected Dashboard

Description: A page only accessible to authenticated users.

Requirements:

- Route: `/dashboard`
- Must check for `auth-token=logged-in` cookie
- If not authenticated → redirect to `/login`
- Display welcome message: "Welcome! You are logged in."
- Provide logout button (red, labeled "Logout")

Test Scenarios:

- Authenticated user accesses dashboard → show content
 - Unauthenticated user accesses dashboard → redirect to login
 - Click logout → clear cookie, redirect to login
-

3.3 Contact Form

Description: A form to submit contact inquiries.

Requirements:

- Route: `/contact`
- Fields:
 - Name (required, text input)
 - Email (required, email input)
 - Message (required, textarea, 4 rows)
- Client-side validation:
 - All fields required
 - Show error: "Fill all fields correctly" if validation fails
- On submit:
 - POST data to `/api/contact`
 - On success (200): Show "Message sent!" (green)
 - On error: Show "Fill all fields correctly" (red)
- Form styling: Max width 384px, flex column layout

Test Scenarios:

- Submit with all fields → success message
 - Submit missing fields → error message
 - API returns error → show error message
-

3.4 API Routes

3.4.1 GET `/api/hello`

Requirements:

- Method: GET
- Response: JSON `{ "message": "Hello from TestSprite API!" }`

- Status: 200

Test Scenarios:

- GET request → returns correct JSON
- Response status is 200

3.4.2 POST /api/contact

Requirements:

- Method: POST
- Expected body: `{ "name": string, "email": string, "message": string }`
- Validation: All fields required
- Success: Return `{ "success": true, "message": "Message received" }` (status 200)
- Error: Return `{ "error": "Missing required fields" }` (status 400)

Test Scenarios:

- POST with valid data → success response
 - POST missing fields → 400 error
 - POST with empty strings → 400 error
-

3.5 Navigation & Routing

Description: Consistent navigation across all pages.

Requirements:

- Navigation bar present on all pages
- Location: Top of page, gray background (`bg-gray-100`)
- Links (in order):
 - Home (`/`)
 - About (`/about`)
 - Contact (`/contact`)
 - Login (`/login`)
 - Dashboard (`/dashboard`)
- All links functional and navigate correctly

Test Scenarios:

- Click each nav link → correct page loads
 - Navigation present on all pages
 - Active state handling (if implemented)
-

3.6 About Page

Description: Static informational page.

Requirements:

- Route: `/about`
- Display heading: "About This Demo" (2xl, bold)
- List features:
 - Login with test credentials
 - Protected dashboard page

- Contact form with API integration
- Navigation across multiple pages

Test Scenarios:

- Page loads correctly
 - All content visible
 - Navigation works from this page
-

3.7 Home Page

Description: Landing page introducing the demo.

Requirements:

- Route: `/`
- Heading: "Welcome to TestSprite Demo" (3xl, bold)
- Paragraph explaining the app's purpose
- Bullet list of testable features:
 - Authentication flows (Login/Logout)
 - Form validation and submission
 - API routes (GET and POST)
 - Protected pages (Dashboard)
 - Navigation and routing
 - Error handling and 404 pages

Test Scenarios:

- Page loads correctly
 - All features listed
 - Navigation to other pages works
-

3.8 Error Handling

Description: Handle invalid routes gracefully.

Requirements:

- Custom 404 page at `app/not-found.tsx`
- Display: "404 - Page Not Found"
- Subtitle: "The page you are looking for does not exist."
- Link back to home: "Go Home" (blue, underlined)
- Any invalid route → show 404 page

Test Scenarios:

- Navigate to `/nonexistent` → shows 404 page
 - Click "Go Home" → redirects to `/`
 - Direct URL access to invalid route → 404 page
-

4. Technical Requirements

4.1 Technology Stack

- **Framework:** Next.js 16.x (App Router)
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **Runtime:** Node.js ≥22

4.2 Project Structure

```
testsprite-demo/
├── src/app/
│   ├── layout.tsx      # Root layout with navigation
│   ├── page.tsx        # Home page
│   ├── login/page.tsx  # Login page (client component)
│   ├── dashboard/page.tsx # Protected dashboard (client component)
│   ├── contact/page.tsx # Contact form (client component)
│   ├── about/page.tsx  # About page (server component)
│   ├── api/hello/route.ts # GET API endpoint
│   ├── api/contact/route.ts # POST API endpoint
│   └── not-found.tsx   # Custom 404 page
├── README.md           # Project documentation
└── package.json
```

4.3 Cookie Management

- Auth cookie name: `auth-token`
 - Auth cookie value: `logged-in`
 - Cookie path: `/`
 - Logout: Set expiry to past date
-

5. Test Coverage Requirements

TestSprite should generate and execute tests covering:

5.1 Functional Testing

- ☐ Login flow (success + failure)
- ☐ Logout flow
- ☐ Protected route access (authenticated vs unauthenticated)
- ☐ Contact form submission (success + validation errors)
- ☐ Navigation between all pages
- ☐ 404 page for invalid routes

5.2 API Testing

- ☐ GET `/api/hello` returns correct response
- ☐ POST `/api/contact` with valid data returns success
- ☐ POST `/api/contact` with invalid data returns 400

5.3 UI/UX Testing

- ☐ All pages render correctly
- ☐ Form inputs accept and validate data

- ☐ Buttons trigger correct actions
- ☐ Navigation bar present on all pages

5.4 Edge Cases

- ☐ Direct URL access to protected routes without auth
- ☐ Form submission with empty fields
- ☐ API calls with malformed JSON
- ☐ Cookie expiration handling

6. Test Credentials

For all test scenarios requiring authentication:

Field	Value
Email	test@example.com
Password	password123

7. Success Criteria

The application is considered complete when:

1. All pages render without errors
2. Login works with test credentials
3. Dashboard is protected and requires auth
4. Contact form submits and calls API correctly
5. API routes return expected responses
6. Navigation works across all pages
7. 404 page displays for invalid routes
8. Logout clears session and redirects

8. Out of Scope

The following are NOT included in this demo:

- Real database integration
- Email sending functionality
- User registration
- Password reset
- Production deployment configuration
- Advanced error tracking
- Analytics

Appendix A: Test Instructions for TestSprite

When uploading this PRD to TestSprite:

1. Select "Next.js" as project type

2. Set test scope to "codebase" (full project)
3. Provide test credentials: `test@example.com` / `password123`
4. Set application URL to `http://localhost:3000` (or Vercel preview URL)
5. Enable testing for: UI flows, API routes, authentication, forms, error handling

End of PRD